Data Science For Life Scientist

Introduction to deep learning: DNNs and transformers



Amina Mardiyyah Rufai

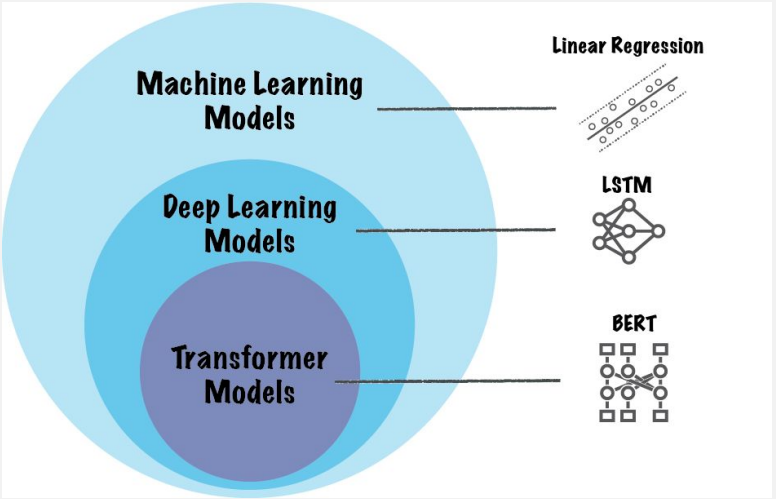
Machine learning Data Scientist

EMBL-EBI

Part A: Introduction to DNNs

What we will cover

- ➔ What deep learning is, and how it differs from traditional machine learning
- ➔ The structure of a neural network: An artificial neuron, weights, biases, activation functions, layers
- ➔ What makes a model “deep”: Hierarchical representation learning
- ➔ The supervised learning workflow: inputs, labels, training, validation, testing



Before we start: quick audience check

https://ahaslides.com/I1893



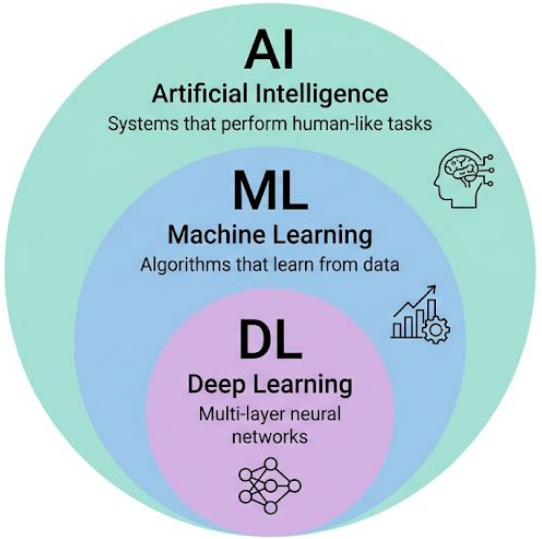
EMBL-EBI

Deep Learning in Context

Deep learning is a subset of machine learning that uses neural networks with multiple layers to learn useful representations directly from data.

Key idea: Representation Learning

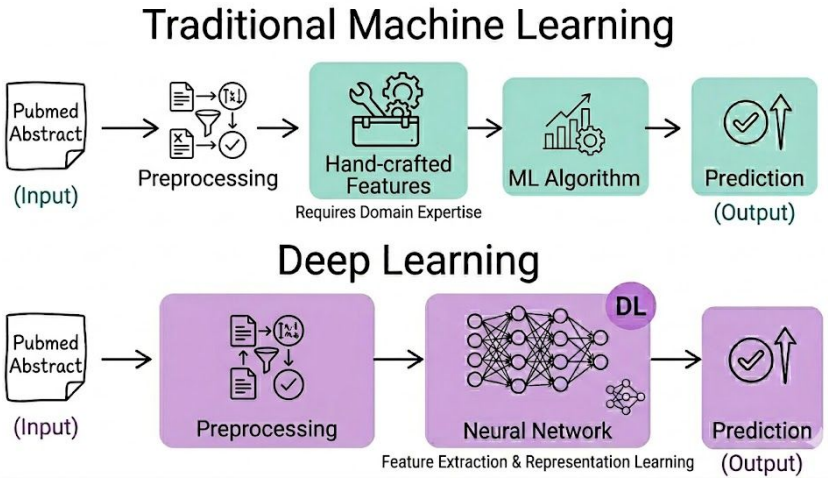
The model learns internal encodings of the data, reducing — but not eliminating — the need for manual feature engineering.



Deep Learning ⊂ Machine Learning ⊂ Artificial Intelligence

Deep Learning Vs Machine Learning: The workflow

- Traditional machine learning
  - Humans design useful input features
  - The model learns from those features
- Deep learning
  - The model learns useful representations from the data; reducing the need for manual feature engineering



Traditional ML vs Deep Learning: Key differences

Aspect	Traditional machine learning	Deep learning
Input data	Often strong on structured/tabular data	Strong on text, images, audio/video, sequences; can also use tabular data
Feature engineering	Relies more on hand-crafted features	Learns useful representations directly from data
Typical data needs	Can work well with smaller datasets	Often benefits from larger datasets or pretrained models
Compute/resources	Usually lower compute demand	Usually higher compute demand; GPU often helpful
Interpretability	Often easier to inspect and explain	Often harder to interpret directly
Life-science examples	Clinical risk prediction, tabular omics, TF-IDF text classification, etc	Biomedical NLP, protein/DNA sequence modelling, etc

Anatomy of a Neural Network

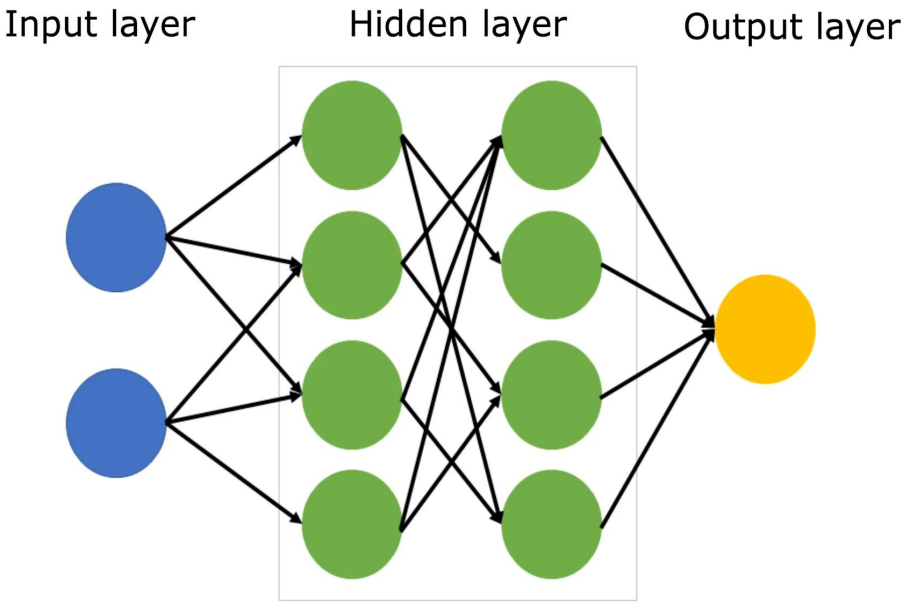
Layered representation learning, loosely inspired by the brain



The DL architecture: Inside a Neural Network

- A "**neural network**" is a machine learning model made of connected layers that learn useful patterns and representations from data to make predictions.

- **Structure of a neural network**
  - **Input layer**: receives the input data
  - **Hidden layers**: learn useful internal patterns and representations
  - **Output layer**: produces the final prediction



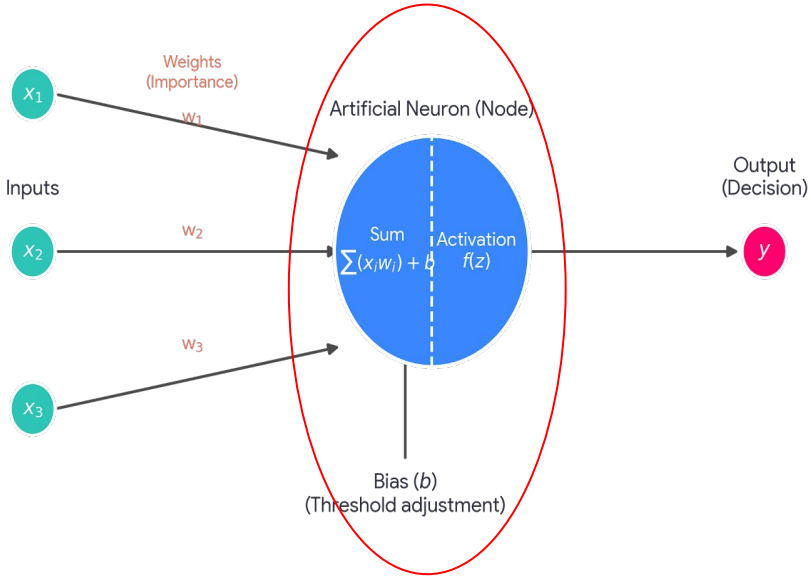
Artificial neural networks

For biomedical NLP, the input is usually a numerical representation of text.

Inside a Neural Network: An artificial neuron

- An artificial neuron is the basic building block of a neural network.

- High-level summary of its primary steps:
  - Receives several pieces of information(features) from the data as inputs
  - Uses learned weights to give different importance/relevance to different inputs(Not all inputs are equally important)
  - Adds these up for a weighted sum
  - Adds the weighted inputs together with a bias term
  - Passes the weighted sum through a decision-filter called the “activation function”
  - Produces an output



EMBL-EBI

An artificial neuron: Inputs, weights and bias

A neuron starts by turning input values into a raw score.

$$z = \Sigma(x_i w_i) + b$$



Inputs (x_1, x_2, x_3, ……, x_n):

Numerical features or signals entering the neuron. In a toy image recognition example, this could be: fur texture, whiskers, nose shape, ear outline.

Weights (w_1, w_2, w_3, ……, w_n)

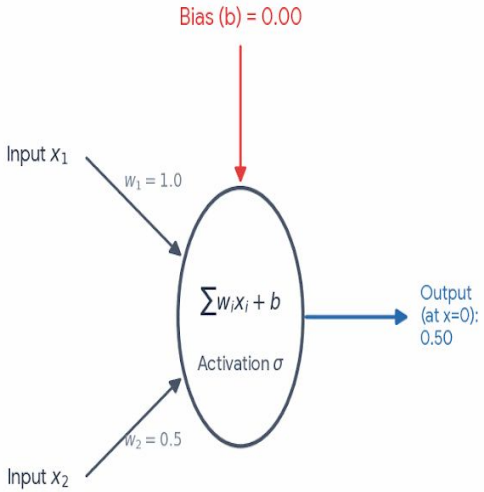
Learned values that control how strongly each input influences the neuron's output.

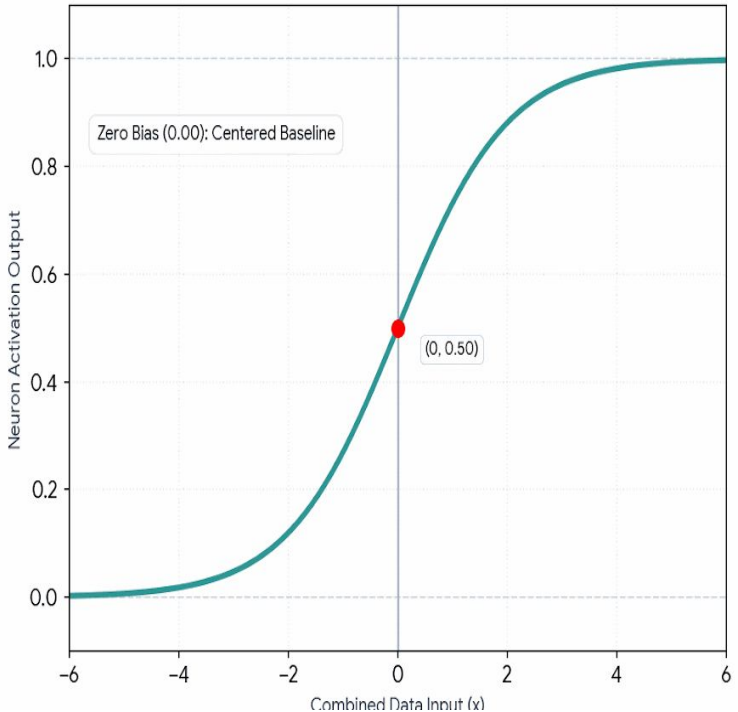
Bias (b)

An extra learned value added to the weighted sum. It shifts the activation and helps the model fit patterns more flexibly.

Training learns w and b. Prediction uses them.

An artificial neuron: Inputs, weights and bias





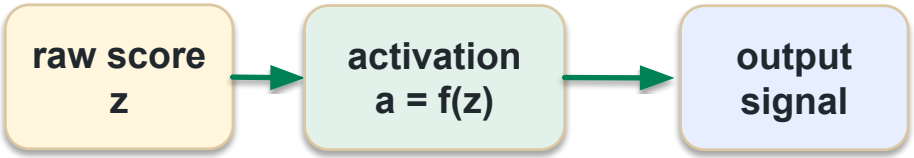
Visualising the effect of bias on an artificial neuron

EMBL-EBI

An artificial neuron: Activation functions

The activation function(**σ**) transforms the raw score into an output signal. So far, we know that in the artificial neuron:

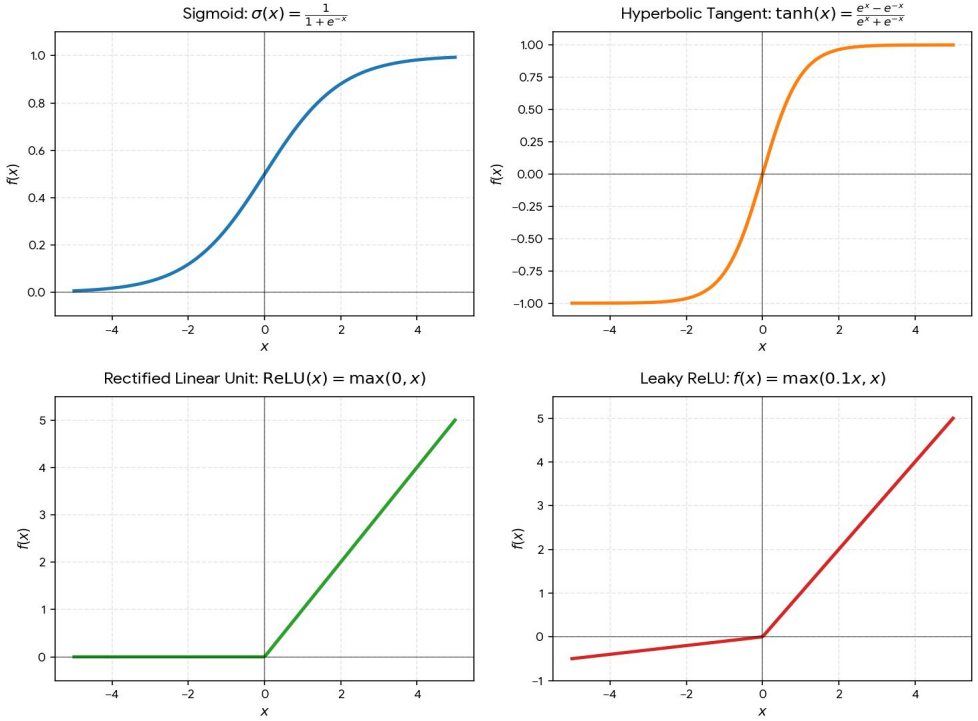
- ➔ The neuron calculates the weighted sum of its inputs plus the bias:
- ➔ The activation function takes this raw total (z) and squashes it into a predictable range:



- ➔ Introduces non-linearity, so networks can learn complex relationships
- ➔ Without non-linear activations, stacked layers behave like a linear model

In practice, activation choice affects training behaviour and how signals move through the network.

An artificial neuron: Activation functions



ReLU
max(0, z)
common in
hidden layers

Sigmoid
0 to 1
binary
probability

Tanh
−1 to 1
centred signal

Not all activations "squash" values: ReLU is unbounded above

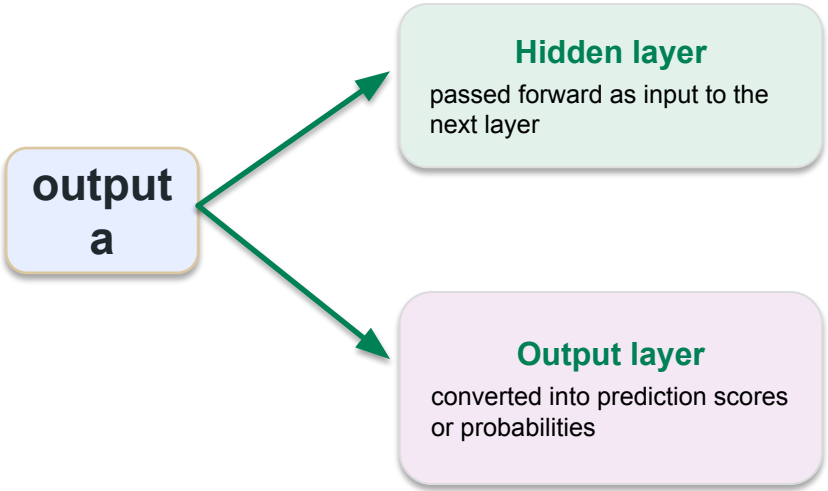
Common Activation functions used in Deep learning models

EMBL-EBI

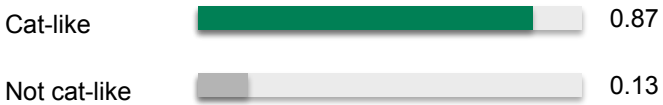
An artificial neuron: The output

After activation, the neuron produces an output value.

- ➔ In hidden layers, this output becomes input to neurons in the next layer
- ➔ In the final layer, outputs become class scores or probabilities
- ➔ For classification, the largest probability is often used as the predicted class

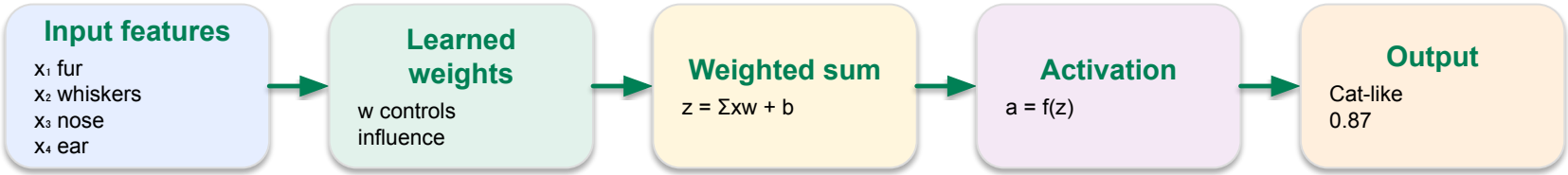


Toy binary prediction



An artificial neuron: from inputs to output

Toy image-recognition example

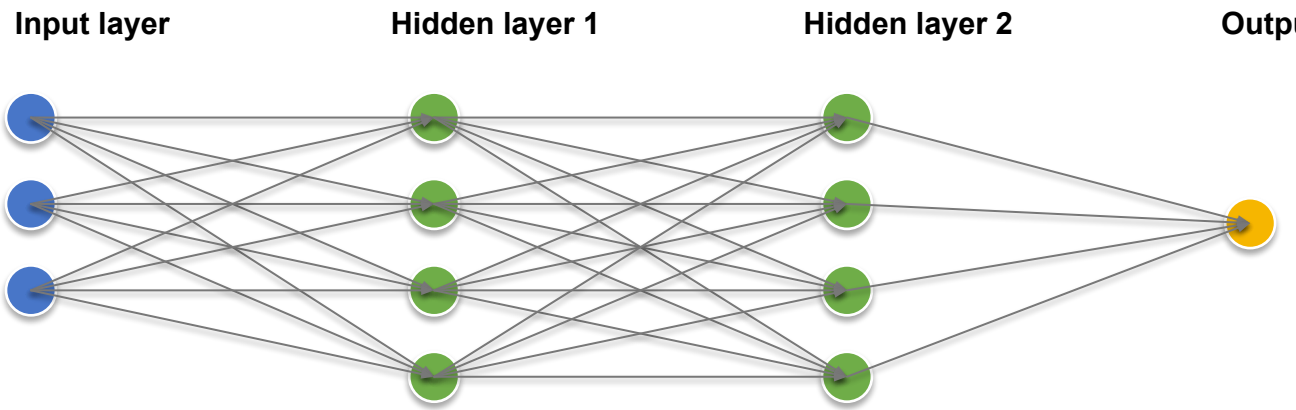


Key idea

- ✓ During training, the model learns the weights and bias
- ✓ During prediction, the neuron uses those values to transform inputs into an output signal
- ✓ A single neuron is simple; useful behaviour emerges when many are connected in layers

Inside a Neural Network: From neurons to layers

Neurons become useful when they are arranged into layers. The output from one layer becomes the input to the next.



Input

raw or vectorized data

Simple patterns

early hidden layer

Richer representations

deeper hidden layer

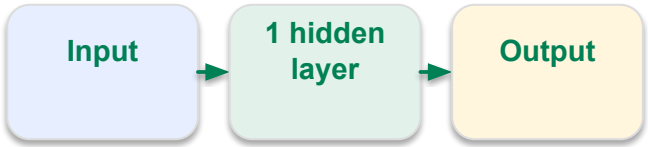
Prediction

output layer

What makes a DL model deep?

A DL model is called “deep” when it has multiple hidden layers that learn increasingly useful representations of the data

A shallow model



A deep model

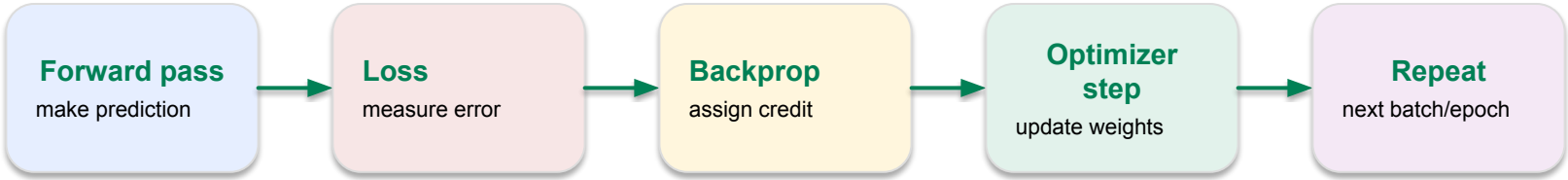


The heart of deep learning is “hierarchical representation learning”

- ✓ Earlier layers often capture simpler signals; later layers combine them into richer patterns
- ✓ More depth increases model capacity — but also training complexity and resource needs
- ✓ Deeper is not automatically better: data quality and evaluation still matter

Training a DL model: How does a neural network learn?

Training is an iterative loop: predict, measure error, update, repeat.



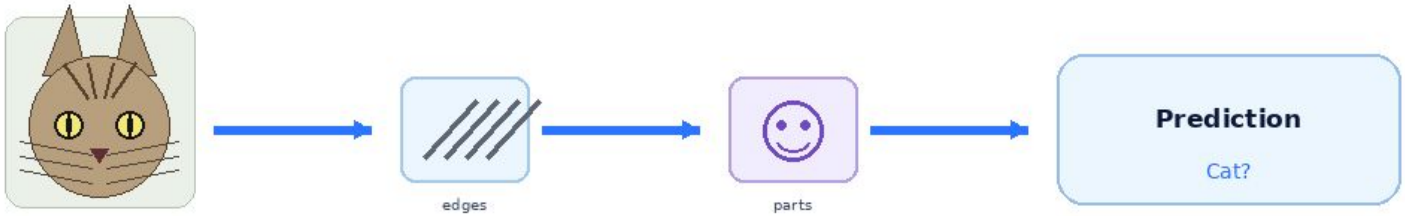
- ✓ The model starts with imperfect parameter values
- ✓ The loss tells the model how wrong its prediction was
- ✓ The optimizer adjusts weights and biases to reduce future error
- ✓ Learning curves show whether training is improving or overfitting/underfitting

Hierarchical Representation Learning

A full training loop: forward pass -> loss -> backpropagation -> updated representations

What this GIF shows

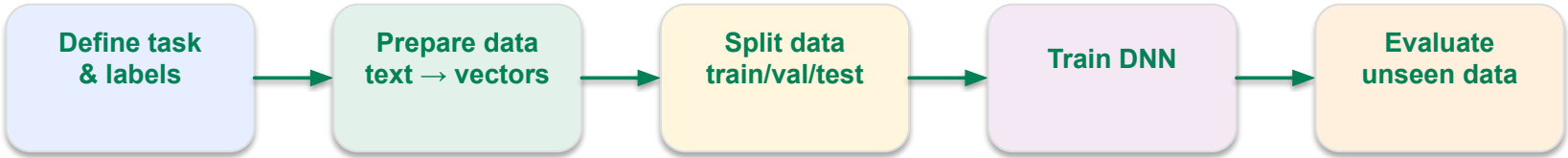
A neural network learns hierarchical representations. Early layers learn simple visual patterns, middle layers combine them into motifs, deeper layers form object parts, and the output layer predicts a class. Training compares the prediction with the true label, sends the error backward, and updates weights.



Goal: learn features that make the correct prediction more likely.

Supervised DNN workflow: from input to output

In supervised learning, the model learns from examples where the correct answer is known.



Biomedical NLP example

- Input: title/abstract, sentence, or clinical-style text
- Label: topic, relevance, document class, or entity class
- Output: predicted label or probability
- Metrics: accuracy, precision, recall, F1 — choose based on task and class balance

Common DL architectures

and where they are useful in life-science data

Different neural-network designs are suited to different data structures and research questions.

Feed-forward DNN / MLP

- Fixed-size feature vectors
- Tabular data or engineered text features
- Good first DNN baseline

CNN

- Images and spatial patterns
- Microscopy and histology
- Learns local filters/features

RNN / LSTM

- Sequential or time-series data
- Older approach for text and sequences
- Captures order step by step

Transformers

- Text and biological sequences
- Use attention to model context
- Often pretrained then fine-tuned

Part A focus: a simple feed-forward DNN. Part B: transformers for contextual sequence modelling.

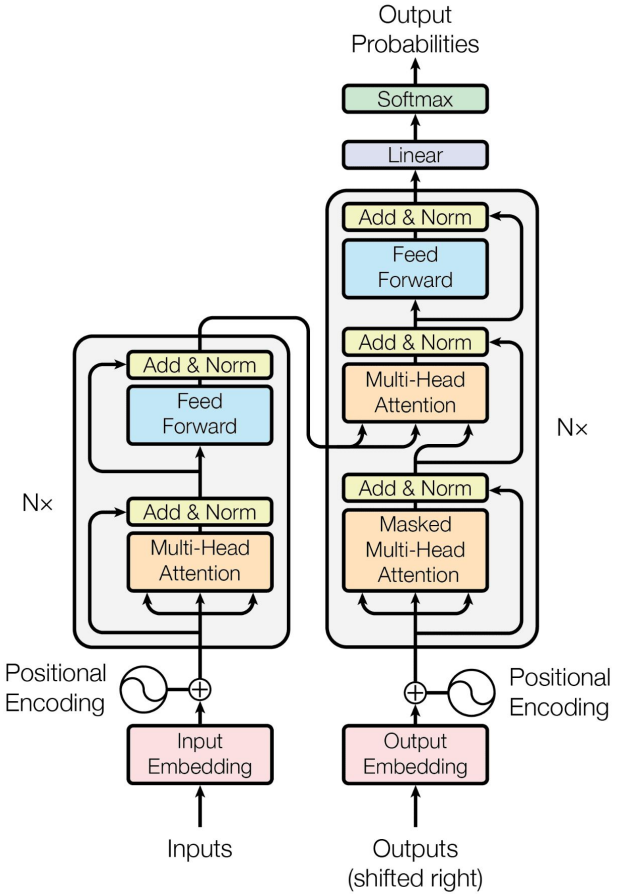
Part A recap: what to take forward

- ★ Deep learning is a subset of ML built around hierarchical representation learning
- ★ The basic unit of an neural network is an artificial neuron
- ★ Artificial neurons combine inputs using learned weights, biases and activations
- ★ ‘Deep’ means multiple hidden layers that learn increasingly useful representations of the data
- ★ During training neural networks learn weights and bias through an iterative process of forward and backward pass.
- ★ Training adjusts model parameters using loss, backpropagation and an optimizer
- ★ An error function is a measure of how well a network is training
- ★ In the prediction phase, the neural networks transform inputs through connected layers and uses the learned weights during training to produce predictions

Next: Transformers — deep learning architectures designed to model context in sequences and text.

Part B: Introduction to Transformers

"Attention is all you need"



From layered representations to contextual representations

EMBL-EBI

Part B: What we will cover

While Part A was focused on neural-network foundation. Part B is about modern DL architectures specialised at handling contexts in text/sequential data.

- ✓ What is sequence modelling?
- ✓ From transformers to DNNs: What changes?
- ✓ How transformers use tokenisation, embeddings and position information
- ✓ The intuition behind self-attention and multi-head attention
- ✓ How transformer blocks form deep architectures
- ✓ Pretraining vs Finetuning: Modern Transformer Workflow

Sequence vs Language Modelling

- **Sequence modelling** involves building models for data where “order” matters.
  - Examples: text, timeseries, speech, DNA sequences, Video frames, etc
- **Language modelling** is a subcategory of sequence modelling specialised for natural language or text.
  - Examples: Naive Bayes, n-grams, hidden-markov models

Aspect	Sequence modelling	Language modelling
Scope	Broad	Specific
Data	Any ordered data	Text tokens
Goal	Depends on task	Learning probability distribution over text
Output	Labels, predictions, generated sequence, forecasts, etc.	Text generation, embeddings, next token prediction

All language models are sequence models, but not all sequence models are language models.

Sequential data: Why order and context matter

A sequence is not just a bag of words: order, negation and surrounding terms can change meaning.

Order changes roles

The dog bit the man.
The man bit the dog.

Same words, different event.

Context changes meaning

The river bank flooded.
The bank approved the loan.
You can bank on that

"bank" means different things.

Negation changes interpretation

The patient responded.
The patient did not respond.

One word flips the meaning.

Transformers build token representations that depend on surrounding tokens.

For language, the meaning of a token is rarely independent of its context.

Biomedical context changes entity meaning

Example Usecase: In biomedical NLP, neighbouring words could determine whether a mention is a cell type or a cell line, a gene or a protein

CellType context

"**T cells** were isolated from peripheral blood."

T cells → CellType

CellLine context

"The **T cell line** Jurkat was cultured overnight."

Jurkat / T-cell line → CellLine

Which context?

"reactivate HIV in **CD4+ T cell line** models"

"which are comprised of memory-resting **CD4+ T cells**"

CellLine or CellType?

Why this matters for transformers

- ➔ The model must use surrounding words, not just isolated terms.
- ➔ Entity type, relevance and document meaning often depend on context.
- ➔ Attention gives each token a mechanism to use information from other tokens.

EMBL-EBI

How does Transformers come in the picture?

They were designed to model sequence context efficiently.

The problem

Text/sequential data contain dependencies between elements that may be far apart.

Before transformers

Previous DNN models(E.g RNN, CNN, etc) struggled with long-range context dependency in texts

Transformer idea

Use attention so tokens can exchange information with other tokens directly.

Transformers were introduced to make context modelling more direct, parallel and scalable.

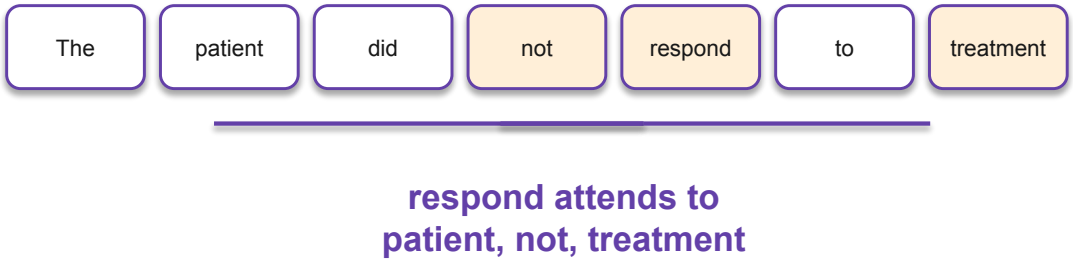
Introduced in the paper: Vaswani et al., 2017, “Attention Is All You Need”.

What is a Transformer?

- ➔ A **transformer** is a deep learning architecture that uses **attention** to learn contextual relationships between tokens in a sequence.
- ➔ It was first introduced in the paper: “Attention is all you need” (Vaswani et al., 2017)

Key idea

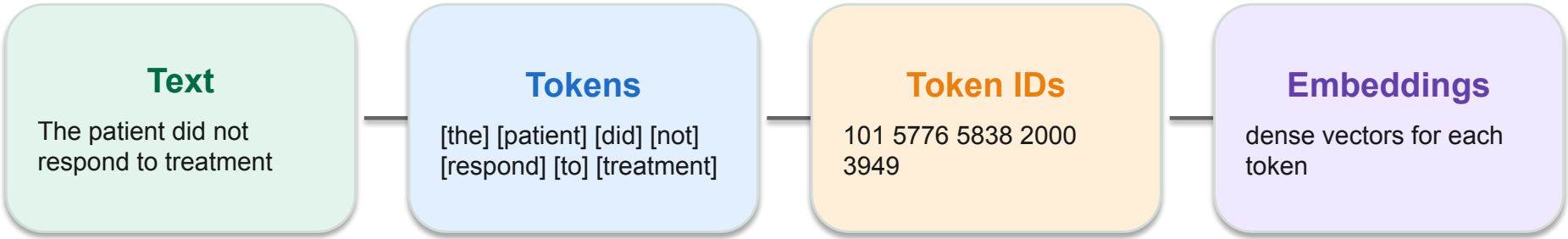
Instead of processing a sequence only step by step, a transformer lets tokens “look at” other tokens to build context-aware representations.



a transformer learns context by letting tokens exchange information.

Transformers Intuitively: Text → tokens → embeddings

NLP models cannot process raw text directly. Text is converted into numerical vectors. Transformers handle this slightly differently from traditional NLP models.

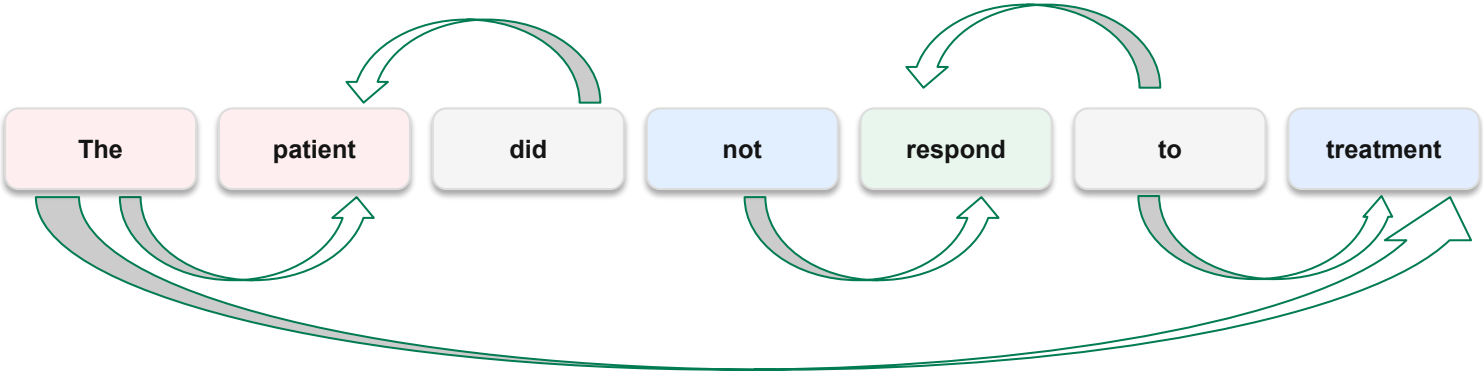


Key concepts

- Tokenization: splitting a text into atomic units for modelling.
- Token IDs: Mapping of each token to a representation in the model's vocabulary
- Embeddings: are dense vectors that represent token meaning and become the transformer's numerical starting point.

Transformers Intuitively: Self Attention at a high-level

sentence: The patient did not respond to treatment



Question attention asks

For this token, which other tokens are most relevant for understanding it?

Multi-head attention: different perspectives

A single attention pattern may capture one relationship. Multiple heads let the model capture several patterns at once.

The patient did not respond to treatment

Head 1

Focus: negation
not → respond

Head 2

Focus: entity context
patient → respond

Head 3

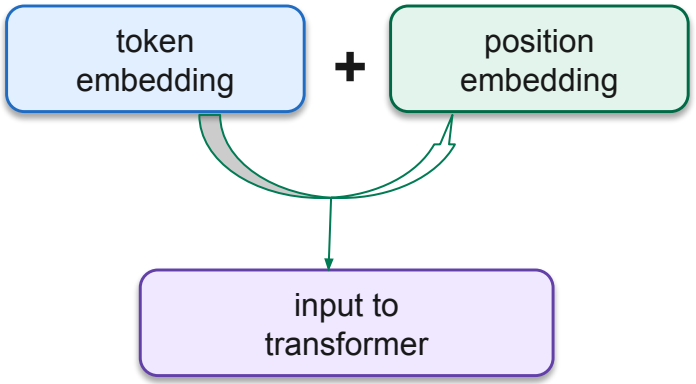
Focus: task context
treatment → respond

Transformers intuitively: adding positional information

Attention compares tokens, but the model also needs to know where each token appears.

Same words, different meaning

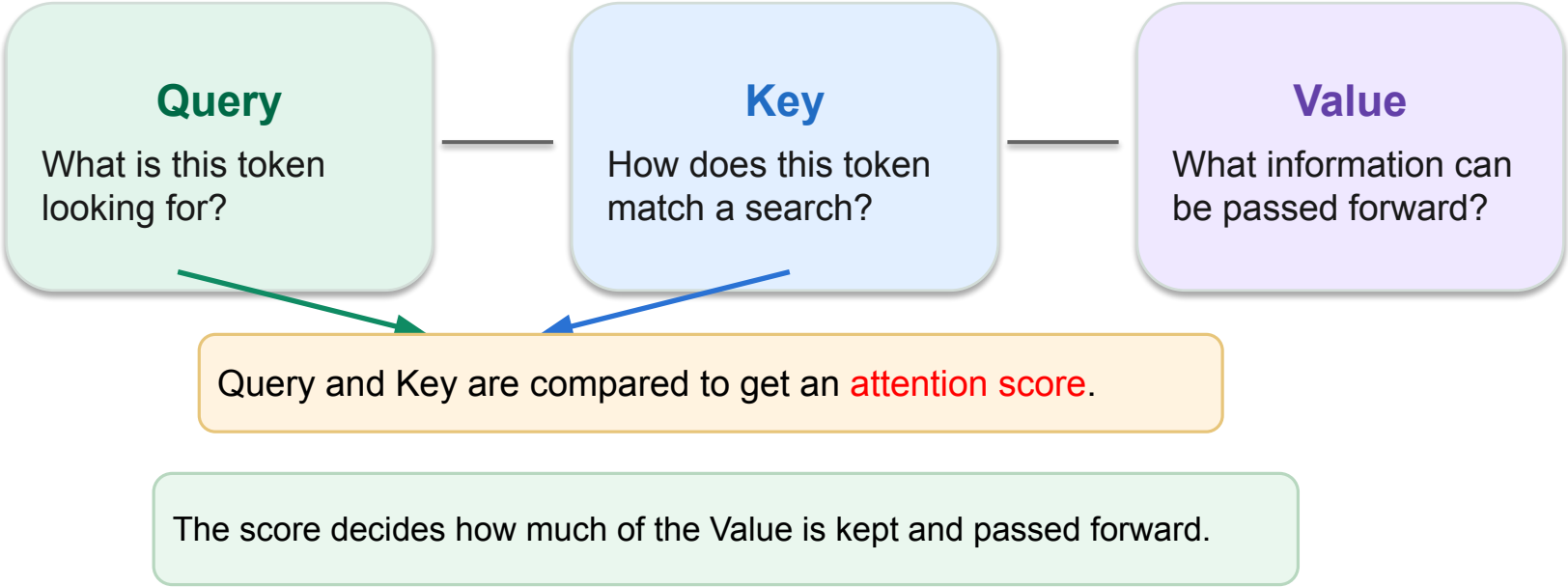
Gene A inhibits Gene B
Gene B inhibits Gene A



Transformers need both context and position.

Transformers Intuitively: Key, Query, Value

For each token, the model learns three vectors that work together in attention.: query, key, and value.



Example: Finding an article/document

Query = Article to find
Key = The catalog/subject topic the article belongs to e.g Cell Biology, Review etc
Value = the actual content inside the article

Transformers intuitively: Encoder vs decoder

Encoders build contextual understanding of an input; decoders generate an output sequence.

E Encoder

Reads the input and builds contextual representations

Purpose: understand the input

- ➔ Uses self-attention over input tokens
- ➔ Best for Natural Language understanding tasks(NLU)

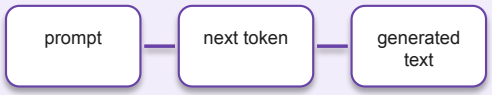


D Decoder

Generates output tokens step by step

Purpose: write/generate output

- ➔ Uses causal attention to avoid seeing future tokens
- ➔ Best for generation tasks

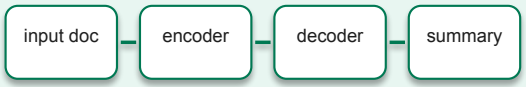


E-D Encoder–Decoder

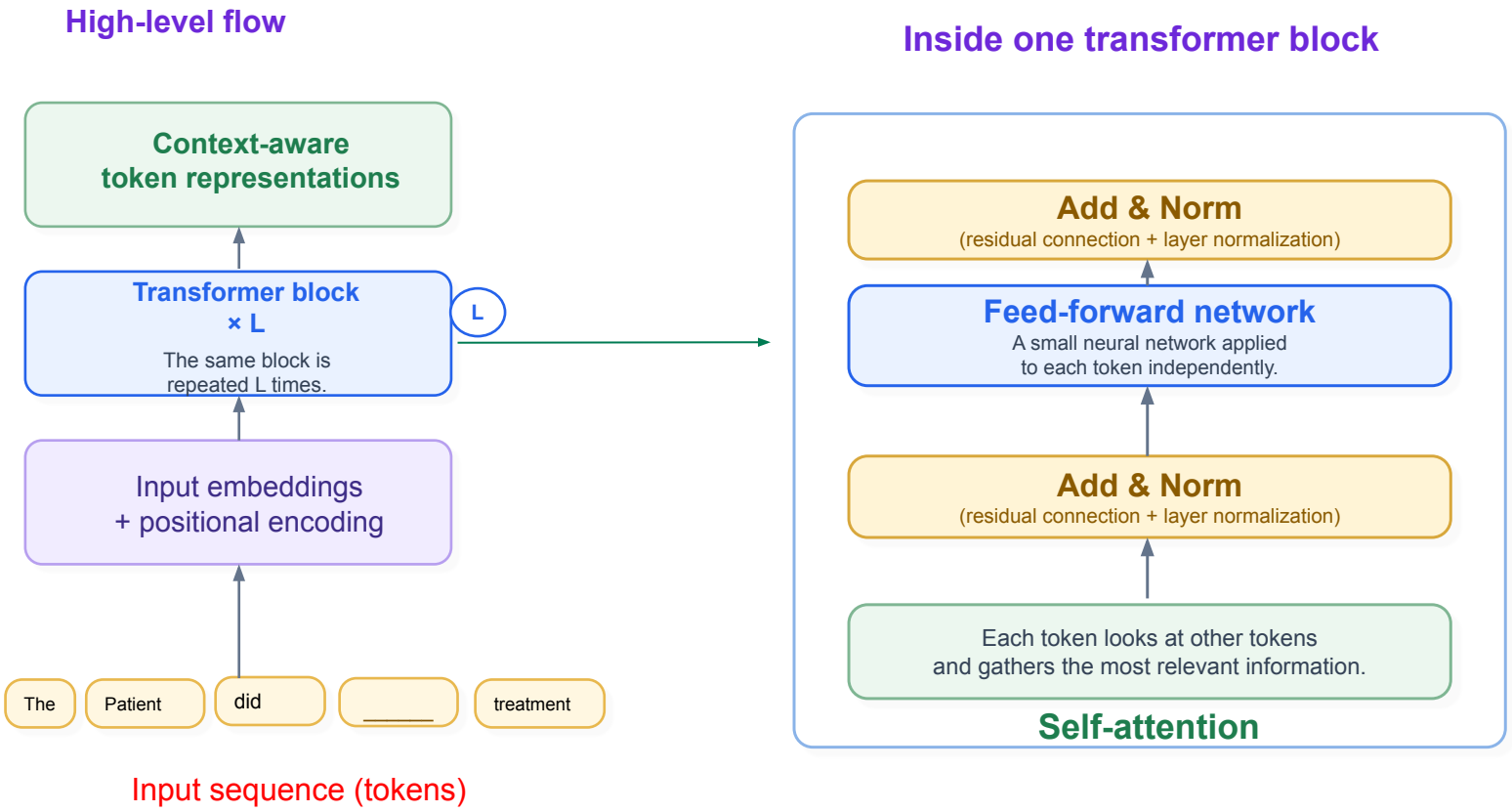
Reads input, then generates a target sequence

Purpose: transform one sequence into another

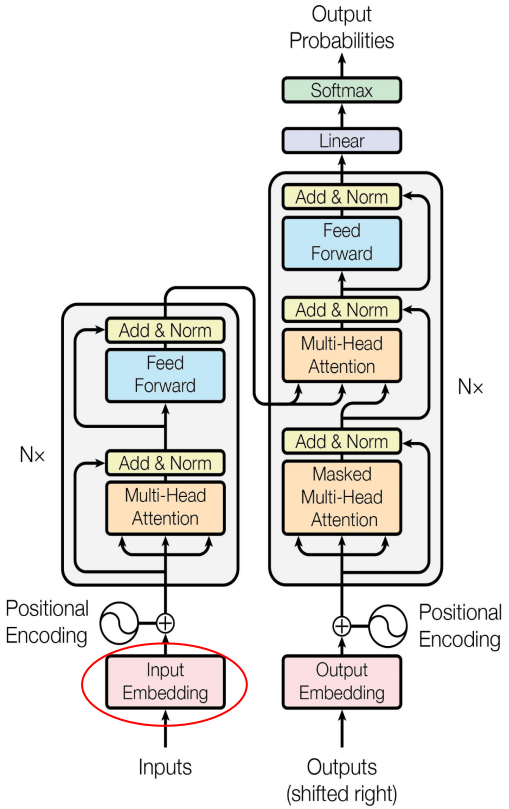
- ➔ Encoder understands the source text
- ➔ Decoder writes the output conditioned on it
- ➔ Best for translation task



Putting it altogether: A Transformer block



Putting it altogether: A Transformer block



Inside one transformer block

Add & Norm
(residual connection + layer normalization)

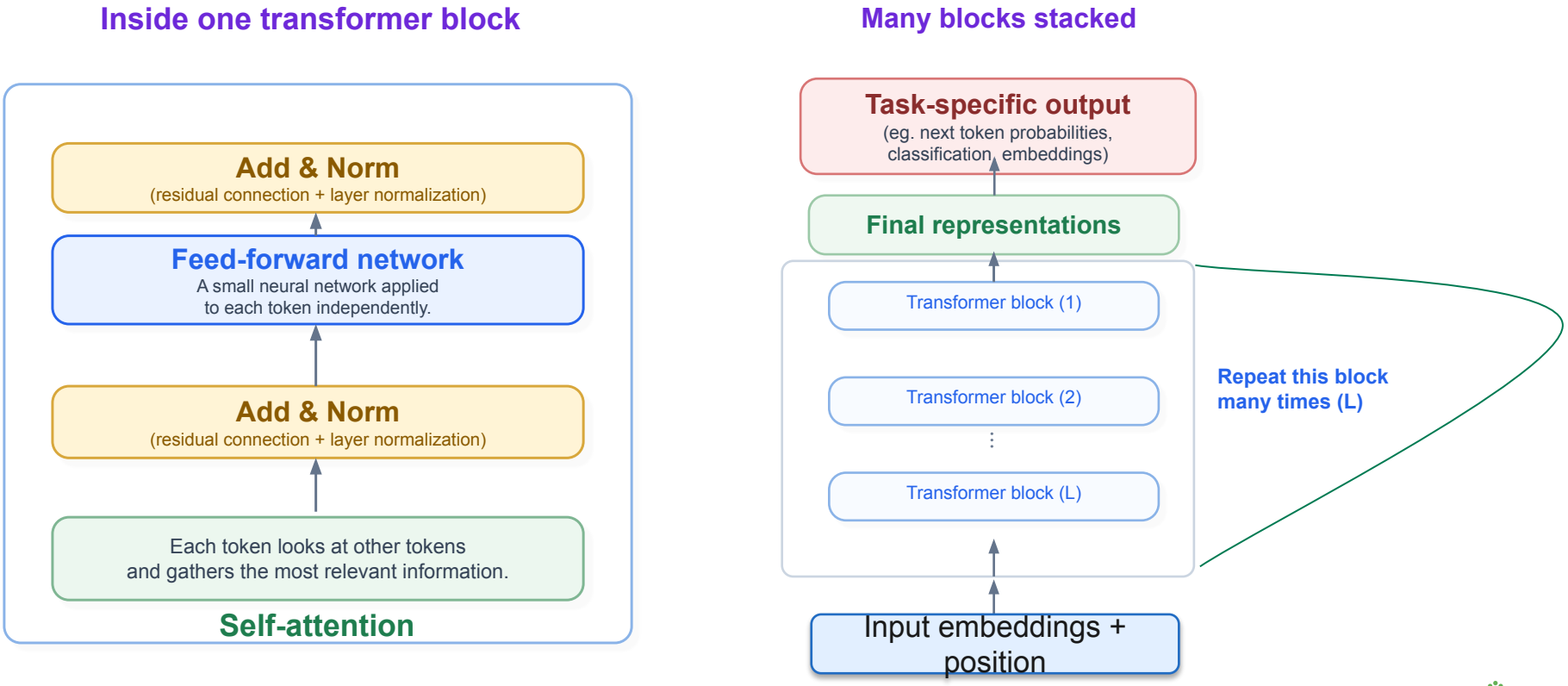
Feed-forward network
A small neural network applied to each token independently.

Add & Norm
(residual connection + layer normalization)

Each token looks at other tokens and gathers the most relevant information.

Self-attention

Putting it altogether: A Transformer block



EMBL-EBI

Transformer families: BERT, GPT and T5/BART

Transformer families differ in how they read input and produce output.

Encoder-only

Reads the full input and builds contextual representations.

Best for: classification, NER, information retrieval

Examples: BERT, BioBERT, PubMedBERT

Decoder-only

Generates text one token at a time, conditioned on previous tokens.

Best for: generation, chat, completion

Examples: GPT-style models

Encoder–decoder

Reads input with an encoder, then generates output with a decoder.

Best for: summarisation, translation

Examples: T5, BART

For biomedical text classification, encoder-only models are usually the cleanest starting point.

Transformer families: when to use each

Different transformer models are best suited for different NLP tasks.

If your output is a label

Use an encoder-only model.

Examples:
- Text classification
- Named entity recognition

Example architectures:
BERT/BioBERT/PubMedBERT

If your output is generated text

Use a decoder or encoder-decoder model.

Examples:
- text generation
- summarisation

If your input and output are both sequences

Use an encoder-decoder model.

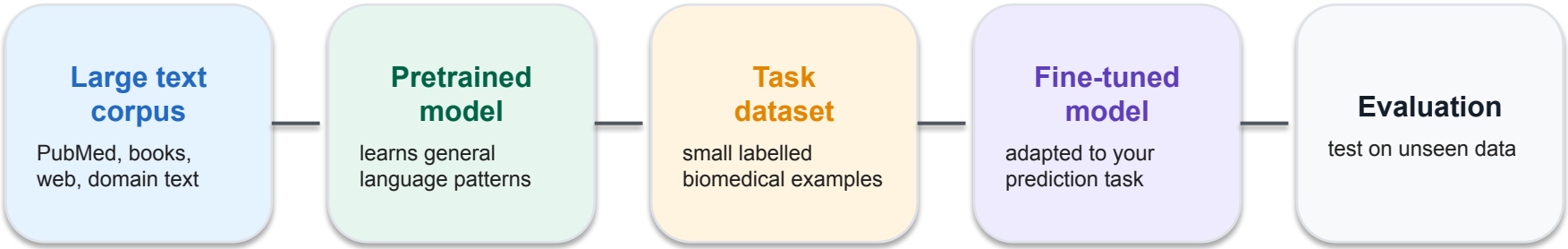
Examples:
- summarisation
- translation

The encoder reads; the decoder writes.

Match the model family to the task: label prediction, token labelling or text generation.

Modern workflow: The pretraining-finetuning paradigm

Most modern transformer workflows start from a pretrained model rather than training from scratch.



Pretraining

Learns reusable language representations from large unlabelled text.

Fine-tuning

Adapts the pretrained model to a labelled task such as classification or NER.

Fine-tuning reuses learned language knowledge, then specialises it for your biomedical task.

Simple feed-forward DNNs vs Transformers: Are they different?

Concept	Simple DNN from Part A	Transformer
Input	Fixed-size vector for all tokens	Each token gets its own representation and can interact with other tokens
Representation	One vector for the whole example	One vector per token, updated by context
Order	Not handled directly	Added using positional information
Context	Must be encoded in the input features	Learned through attention

Both are deep neural networks

Both are trained with the same basic loop: forward pass → loss → backpropagation → optimizer step.

Part B recap: what to take forward

- ✓ Transformers are deep learning models designed for sequence context
- ✓ Text is converted into tokens and embeddings before modelling
- ✓ Positional information helps the model use order
- ✓ Self-attention lets each token use information from other tokens
- ✓ Multi-head attention captures different relationship patterns
- ✓ Transformer blocks stack attention and feed-forward layers
- ✓ Modern workflows usually rely on pretraining and fine-tuning

Hands-on session:

Biomedical Text classification with a simple DNN and a Pretrained transformer

Part A: Link
Part B: Link